

Evaluation of RAG Retrievers

1. Session Context & Roadmap for RAG Evaluation

Today's session directly builds upon our previous foundational roadmap, where we established the complete blueprint for evaluating Retrieval Augmented Generation (RAG) applications. Going forward over the next three classes (including this one), our primary objective is to build a complete **RAG Evaluation Suite (RAG Eval Suite)**.

To build a robust evaluation pipeline, we evaluate our RAG application across **three distinct evaluation levels**:

1. **Component Level Evaluation:** Testing individual, isolated modules of the RAG pipeline (specifically the retriever and the generator).
2. **Pipeline/Workflow Level Evaluation:** Testing the end-to-end orchestration flow combining the retriever and generator components.
3. **Application Level Evaluation:** Evaluating the entire user facing system (including UI interactions and application logic).

Collectively, this multi layered evaluation pipeline constitutes our **eval suite**, which serves as our primary framework for **regression testing** whenever we update or modify our application code.

Our focus today is completing the **Component Level Evaluation**. In any standard RAG architecture, two primary components dictate system performance:

- **The Retriever:** Given an input user query, its sole responsibility is to fetch the most relevant context chunks from a vector database.
- **The Generator:** Given the input user query along with the retrieved context, its responsibility is to produce a coherent, relevant, and grounded answer.

Our plan for today is to first construct our retriever component from scratch, establish its evaluation framework, measure its baseline performance, and systematically optimize its operational metrics before moving on to the generator component in subsequent sessions.

2. Standardized Project Setup & Directory Architecture

Before writing application logic or evaluation scripts, we must establish a standardized directory structure to ensure clean separation of concerns across software and evaluation artifacts. Rather than creating files arbitrarily, we organize our project inside a dedicated directory named `rag_eval_project`.

```
rag_eval_project/
```

```
└─ data/      # Stores lecture transcripts serving as the knowledge base
└─ src/       # Application code (retriever, generator, pipeline, UI)
└─ evals/    # Evaluation scripts for component, pipeline, and app levels
└─ goldens/  # Stores curated golden datasets for evaluation
```

Directory Breakdown

- **data/**: Serves as our primary **knowledge base**. Here, we store raw lecture transcripts across our past eight sessions. Each transcript is stored in WebVTT format (`.vtt`), structured similarly to movie subtitles containing time stamps and raw text spoken by the instructor and students.
- **src/**: Contains core application modules, including document loaders, chunking logic, vector store setup, retriever instances, generator pipelines, and Streamlit frontend code.
- **evals/**: Houses our evaluation pipelines, including scripts for evaluating the retriever, the generator, and end-to-end RAG workflows.
- **goldens/**: Contains our reference benchmark datasets (**golden datasets**) used during evaluation runs.

Virtual Environment & Dependency Management

To avoid global dependency conflicts, we manage project dependencies locally using **uv** with **Python 3.11**.

Our application and evaluation suite require the following specific python packages:

- **langchain**: For orchestrating document loading, text splitting, and vector store retriever integrations.
- **openai**: For accessing embedding models (e.g., `text-embedding-3-small`, `text-embedding-3-large`) and judge models.
- **deepeval**: Our primary evaluation framework for running contextual evaluation metrics.
- **pytest**: Required underlying dependency for executing DeepEval test suites.
- **python-dotenv**: For loading environment variables (such as `OPENAI_API_KEY`) from `.env` files.

3. Building the Retriever Component (`src/retriever.py`)

A fundamental rule of LLM application development is that **you must never build the full application end-to-end before testing it**. Instead, we follow strict software engineering practices, write a discrete module, evaluate it in isolation, optimize it, confirm its reliability, and only then proceed to downstream components.

Mechanics of a RAG Retriever

A retriever acts as an isolated function:

1. It accepts an input textual **query**.
2. It converts the query into a dense vector embedding using a specified **embedding model**.
3. It performs a similarity search against a **vector database** containing document text chunks stored as vector embeddings.
4. It extracts and returns the top (K) nearest vector chunks as the **retrieved context**.

```
[Query] —> [Embedding Model] —> [Vector Search in Chroma DB] —> [Top K Context Chunks]
```

Implementation Details

We implement our complete retriever lifecycle inside `src/retriever.py` across three core functions:

1. Document Loading and Preprocessing (*load_transcripts*)

- We scan the `data/` directory and parse all `.vtt` transcript files line by line.
- **Timestamp Filtering:** Raw WebVTT files contain timestamp lines intermingled with transcript text. **We explicitly filter out all timestamp lines.** Keeping raw timestamps in document chunks heavily degrades semantic embedding quality because artificial numbers and time sequences disrupt continuous semantic meaning.
- **Metadata Extraction:** As clean text lines are extracted into LangChain Document objects, we extract and attach session metadata (e.g., Session 1, Session 8) to each document. This enables precise source attribution and citations in downstream responses.

2. Vector Store Ingestion (*load_store*)

- We initialize OpenAI's **text-embedding-3-small** as our default embedding model.
- We establish a persistence directory named `chroma_store` for our **Chroma vector database**.
- The function checks if `chroma_store` already exists on disk:
 - If it exists, it loads the persisted vector database directly to avoid redundant embedding calls.
 - If it does not exist, it loads clean transcript documents, executes document chunking, embeds the chunks, and persists them into Chroma.
- **Initial Chunking Strategy:** We set an initial **chunk_size of 750 characters** and a **chunk_overlap of 100 characters**.

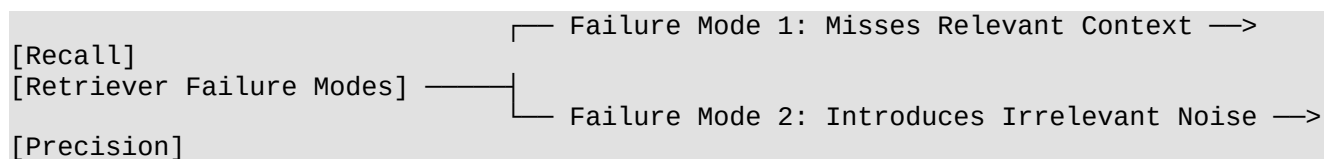
3. Retriever Creation & Verification (*build_retriever*)

- Converts the loaded Chroma vector store into a retriever object using `as_retriever(search_kwargs={"k": 5})` to fetch the top 5 context chunks per query.
- **Smoke Testing:** Running an initial query "What is regression testing?" initializes `chroma_store`, embeds all transcript chunks, and successfully retrieves the top 5 chunks containing relevant explanations from prior sessions.

Handling Document Updates: If a new lecture transcript (e.g., Session 10) is added to `data/`, our setup currently requires deleting the persisted `chroma_store` directory and re-executing `retriever.py` to re-chunk and re-embed the knowledge base. While decoupling data ingestion pipelines from retrieval services is standard for production systems, keeping ingestion local simplifies our primary focus: evaluation methodology.

4. Retriever Failure Modes & Core Evaluation Metrics

To evaluate a retriever rigorously, we must first analyze how a retriever fails in production. A retriever operates as a query in, context out component. Consequently, there are **two fundamental failure modes** for any retriever:



1. Failure Mode 1: Missing Relevant Context

- The retriever fails to fetch the necessary chunks required to answer the query correctly.
- *Example:* Suppose a query requires information residing in chunks `["What is regression testing?", "Regression testing is a type of software testing"]`. If the retriever returns chunks `["Regression testing is a type of software testing", "Regression testing is a type of software testing"]`, it completely misses the required knowledge, making accurate downstream generation impossible.

2. Failure Mode 2: Returning Noisy / Irrelevant Context

- The retriever successfully fetches correct chunks, but includes irrelevant, noisy chunks alongside them.
- *Example:* For a query requiring chunks `["What is regression testing?", "Regression testing is a type of software testing"]`, the retriever fetches `["What is regression testing?", "Regression testing is a type of software testing", "Regression testing is a type of software testing"]`. While relevant context is present, passing irrelevant noise to the generator increases the risk of hallucination or context dilution.

Primary Quantitative Metrics

From these two failure modes, we derive our two core quantitative evaluation metrics:

Recall

- **Definition:** Measures the proportion of relevant context chunks present in the vector store that were successfully retrieved.
- **Mathematical Concept:**
$$\text{Recall} = \frac{\text{Number of Relevant Contexts Retrieved}}{\text{Total Relevant Contexts Available in Vector DB}}$$
- If 5 relevant context chunks exist in the vector DB for a question and the retriever brings 3 of them, $\text{Recall} = \frac{3}{5} = 0.60$

Precision

- **Definition:** Measures the proportion of retrieved context chunks that are actually relevant and useful for answering the question.
- **Mathematical Concept:**
$$\text{Precision} = \frac{\text{Number of Useful Contexts Retrieved}}{\text{Total Contexts Retrieved}}$$
- If the retriever brings 5 chunks and only 2 contain useful information while 3 are noise, $\text{Precision} = \frac{2}{5} = 0.40$

The Precision Recall Tension & Trade Off

An ideal retriever achieves both high recall and high precision (approaching 1.0). However, there is an inherent mathematical tension between precision and recall controlled by the hyperparameter (K) (top-(K) retrieved chunks):

- **Increasing (K) (e.g., from 5 to 10):** Increases **Recall** because retrieving more chunks raises the probability of capturing missing relevant context.
- **Impact on Precision:** Increasing (K) introduces extra non relevant chunks, pulling **Precision** down toward zero.

Reference Based vs. Reference Free Evaluation

Retriever evaluation is strictly a **Reference Based Evaluation**:

- **Reference Based Evaluation:** Requires a pre defined reference standard (**golden dataset**) containing target ground truth context or ideal answers against which model outputs are compared.
- **Reference Free Evaluation:** Evaluates model outputs directly without requiring external reference datasets or golden ground truths.

Because calculating retriever recall and precision requires knowing what constitutes correct context for every query, retriever testing cannot be reference free.

5. The Naive Chunk ID Approach vs. The Invariant Ideal Answer Approach

The Naive Chunk ID Method (And Why It Fails)

In traditional information retrieval, retriever evaluation relies on a golden dataset populated with explicitly annotated document/chunk IDs:

Question	Target Chunk IDs
What is regression testing?	72, 89, 100
What is RAG Triad?	73, 90, 91

Calculation under Naive Approach

If the retriever returns chunks 30 for query 1:

- Relevant target chunks present: 72, 89, 100 (3 out of 3 retrieved $\rightarrow \text{Recall} = \frac{3}{3} = 1.0$)
- Total retrieved chunks: 5 (3 relevant, 2 noise $\rightarrow \text{Precision} = \frac{3}{5} = 0.60$)

Why Chunk ID Annotation Fails in RAG Engineering

While widely documented in textbooks, **this naive Chunk ID based method is flawed for RAG systems:**

1. **Exhaustive Human Labor:** Annotating chunk IDs across 800+ transcript chunks for 50+ questions requires reading all 800 chunks repeatedly per question an unscalable manual task.
2. **Parameter Instability (The Fatal Flaw):** Suppose baseline recall is 65% and we optimize performance by tuning chunking parameters (e.g., changing `chunk_size` from 750 to 1000 characters and `chunk_overlap` from 100 to 150). Changing chunk parameters causes Chroma to generate an entirely new set of 700 re-segmented chunks. Consequently, **every single chunk ID in the golden dataset becomes completely invalid**. The entire golden dataset must be discarded and re-annotated manually from scratch after every single experiment.

When ID Based Testing Works: Chunk ID datasets are acceptable only when documents are static, clearly separated, non overlapping, and chunking parameters are permanently locked. For interconnected lecture transcripts where chunk sizes are iteratively tuned, ID based golden datasets are unusable.

6. DeepEval's LLM-as-a-Judge Contextual Metrics

To eliminate dataset invalidation, we adopt DeepEval's **LLM-as-a-Judge** evaluation approach using

Invariant Ideal Answers.

The Invariant Golden Dataset Format

Instead of document IDs, our golden dataset stores an **Ideal Answer** for every question: |Question | Ideal Answer| |--| |What is regression testing? | Regression testing verifies that a new version of software outperforms or maintains performance relative to previous versions. It executes an eval suite against the software and can be integrated into CI pipelines. |

- **Ideal Answer Standard:** An ideal answer is **not** fetched from external search engines (e.g., Google); it is constructed strictly from the facts discussed in our transcript knowledge base.
- **Invariance Advantage:** If chunk parameters change, the underlying context in our transcripts remains constant. The ideal answer stays valid regardless of how text is chunked on disk.

[Golden Question + Ideal Answer]

- LLM Judge decomposes Ideal Answer into Atomic Claims
- LLM Judge evaluates retrieved chunks against Atomic Claims

Contextual Recall Mechanics

DeepEval evaluates retriever recall using **Contextual Recall**:

1. The retriever fetches top (K) context chunks for a query.
2. An LLM-as-a-Judge decomposes the **Ideal Answer** into distinct **atomic claims**.
 - *Claim 1:* Regression testing tests if new software versions surpass previous versions.
 - *Claim 2:* It runs an eval suite against software.
 - *Claim 3:* It can be used in CI pipelines.
3. The LLM Judge inspects the retrieved context chunks and checks how many of these atomic claims are explicitly supported by the retrieved contexts.
4. **Formula:** -
$$\text{Contextual Recall} = \frac{\text{Number of Ideal Claims Supported by Retrieved Context}}{\text{Total Number of Claims in Ideal Answer}}$$
 - If all 3 atomic claims are present across retrieved chunks, Contextual Recall is $\frac{3}{3} = 1.0$

Contextual Precision Mechanics & Ranking Sensitivity

Standard precision measures what fraction of retrieved chunks are relevant. An LLM Judge evaluates each retrieved chunk against the question and ideal answer using a system prompt:

"Given the Question, the Expected Answer, and a Retrieved Chunk: Does this chunk contain information that helps produce the expected answer? Reply YES or NO."

The Ranking Awareness Problem

Consider two retrieval scenarios for (K=5) where green represents relevant chunks and red represents noise:

Case A (Optimal):	[Relevant]	[Relevant]	[Noise]	[Noise]	[Noise]
Case B (Suboptimal):	[Noise]	[Noise]	[Noise]	[Relevant]	[Relevant]

- Under standard precision, both Case A and Case B score identical precision ($\frac{2}{5} = 0.40$) because both contain 2 useful chunks out of 5.
- However, **Case A is a far superior retrieval outcome** because relevant chunks appear at ranks 1 and 2, placing relevant information directly at the top of the LLM generator's context window.

DeepEval's Ranking Aware Cumulative Precision Formula

DeepEval solves this using **Contextual Precision**, which computes cumulative precision at each position (k) where a relevant chunk is found:

$$\text{Contextual Precision} = \frac{\sum_{k=1}^K (\text{Precision}@k \times \text{Relevance}_k)}{\text{Total Number of Relevant Chunks in Top } K}$$

Step-by-Step Calculation Comparison

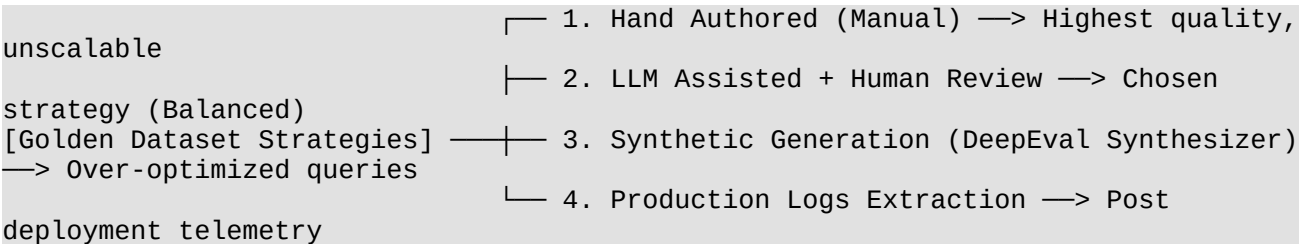
- Case A Evaluation:**
 - Position 1 (Relevant): Cumulative Precision = $\frac{1}{1} = 1.0$
 - Position 2 (Relevant): Cumulative Precision = $\frac{2}{2} = 1.0$
 - Position 3 (Irrelevant): Cumulative Precision = $\frac{2}{3} = 0.66$
 - Position 4 (Irrelevant): Cumulative Precision = $\frac{2}{4} = 0.50$
 - Position 5 (Irrelevant): Cumulative Precision = $\frac{2}{5} = 0.40$
 - Mean Contextual Precision (Case A):** $\frac{1.0 + 1.0 + 0.66 + 0.50 + 0.40}{5} = \mathbf{0.712}$
- Case B Evaluation:**
 - Position 1 (Irrelevant): $\frac{0}{1} = 0.0$
 - Position 2 (Irrelevant): $\frac{0}{2} = 0.0$
 - Position 3 (Irrelevant): $\frac{0}{3} = 0.0$
 - Position 4 (Relevant): $\frac{1}{4} = 0.25$
 - Position 5 (Relevant): $\frac{2}{5} = 0.40$
 - Mean Contextual Precision (Case B):** $\frac{0.0 + 0.0 + 0.0 + 0.25 + 0.40}{5} = \mathbf{0.130}$

By incorporating rank awareness, Contextual Precision heavily penalizes systems that place noise at

top positions.

7. Strategies for Golden Dataset Generation

We analyze four distinct strategies for constructing evaluation golden datasets:



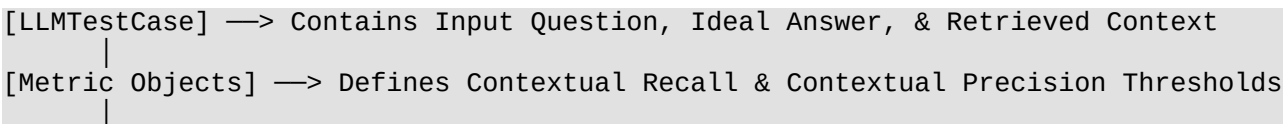
- 1. Hand Authored (Manual):** Domain experts manually write questions and ideal answers.
 - *Pros:* Highest precision and human judgment.
 - *Cons:* Highly labor intensive, costly, and unscalable.
- 2. LLM Assisted Drafting + Human Review (Selected Strategy):** We pass raw lecture transcripts to Claude with explicit instructions to generate student style questions and transcript grounded ideal answers. A domain expert then rigorously reviews and edits every row.
 - *Pros:* Fast, cost effective, preserves natural user query phrasing, and guarantees transcript grounding.
- 3. Synthetic Generation via DeepEval's Synthesizer Module:** Programmatic generation via `goldens/goldens_generator.py` using DeepEval's `Synthesizer` class.
 - *Outcome on our data (retriever_dp_golden.json):* The generated queries were over optimized, academic, and unnatural (e.g., complex GSM8K benchmarks or obscure UPSC questions). Synthetic queries failed to mirror actual student questioning patterns.
- 4. Production Logs:** Extracting real user queries and positive feedback signals (e.g., thumbs up clicks) from live telemetry.

Our Final Benchmark Dataset (`goldens/retriever_goldens.json`)

We selected **Strategy 2**, compiling 15 curated student queries and verified ideal answers into `goldens/retriever_goldens.json`.

8. DeepEval Architecture & Core Primitives

DeepEval operates on three core code primitives:



```
[evaluate()] —> Executes LLM-as-a-Judge evaluation and logs results
```

1. **LLMTestCase**: Represents a single row of the golden dataset.
 - `input`: The question string.
 - `expected_output`: The ideal answer.
 - `retrieval_context`: List of text strings retrieved by the retriever.
 - `actual_output`: Placeholder string (e.g., "Generator not evaluated in this run") since the generator is omitted during retriever component testing.
 2. **Metric Objects**: Configured instances of evaluation metrics.
 - Parameters: `model` (judge model, e.g., `gpt-4o`), `threshold` (pass/fail score bar, set to `0.70`), and `include_reason=True` (logs explicit qualitative justifications).
 3. **evaluate()**: Core execution function that accepts a list of `LLMTestCase` objects and metric instances, running LLM-as-a-Judge calls concurrently.
-

9. Evaluation Pipeline Execution & Iterative Optimization Experiments (`evals/eval_retriever.py`)

Execution Fix for Module Imports

When executing `python evals/eval_retriever.py`, a `ModuleNotFoundError: No module named 'src'` occurs because Python fails to locate `src` relative to the `evals/` execution directory. We resolve this by adding empty `__init__.py` files inside `src/` and `evals/` and running the evaluation module using Python's module flag:

```
python -m evals.eval_retriever
```

Systematic Optimization Experimentation Cycle

We execute five sequential experimentation runs to systematically optimize retriever performance:

```
[Experiment 1: Baseline] —> Recall: 80% | Precision: 80% (5 Failed)
|
[Experiment 2: Chunk Tuning (1000/150)] —> Recall: 97% | Precision: 83% (3 Failed)
|
[Experiment 3: Cross-Encoder Re-ranker] —> Recall: 92% | Precision: 85% (2 Failed)
|
[Experiment 4: Upgrade Embeddings (Large)] —> Recall: 99% | Precision: 85% (3 Failed)
|
[Experiment 5: Top-K Tuning (K=3)] —> Recall: 99% | Precision: 84% (Minor variance)
```

Experiment 1: Initial Baseline Run

- **Configuration:** `chunk_size = 750`, `chunk_overlap = 100`, Embedding Model = `text-embedding-3-small`, (`K = 5`), No Re-ranker.
- **Results:**
 - **Contextual Recall: 80.0%**
 - **Contextual Precision: 80.0%**
 - **Pass/Fail:** 10 Passed, 5 Failed (out of 15 test cases).

Experiment 2: Chunk Size & Overlap Optimization

- **Action:** We increased `chunk_size` from 750 to 1000 characters and `chunk_overlap` from 100 to 150 characters. We deleted `chroma_store` to trigger a clean re embedding run.
- **Observation:** Total chunk count across our transcript corpus dropped from 800 to 697 chunks.
- **Results:**
 - **Contextual Recall: 97.0%** (Significant +17% jump because larger chunks encompass full concepts).
 - **Contextual Precision: 83.0%** (+3% improvement).
 - **Pass/Fail:** 12 Passed, 3 Failed.

Experiment 3: Introducing Cross Encoder Re-ranking (`src/reranker.py`)

- **Hypothesis:** Contextual Precision is ranking aware. Introducing a cross encoder **Re ranker** will re order the top retrieved chunks, pushing highly relevant context to ranks 1 and 2 and forcing noise down.
- **Action:** We implemented a HuggingFace Sentence Transformer cross encoder re ranker inside `src/reranker.py` and wrapped our retriever into `RerankingRetriever`.
- **Results:**
 - **Contextual Precision: 85.0%** (+2% improvement).
 - **Contextual Recall: 92.0%.**
 - **Pass/Fail:** 13 Passed, 2 Failed (highest test case pass rate).

Experiment 4: Embedding Model Upgrade

- **Action:** Upgraded embedding model from `text-embedding-3-small` to OpenAI's larger, higher-dimensional **`text-embedding-3-large`** model. Deleted `chroma_store` and re-embedded.
- **Results:**
 - **Contextual Recall: 99.0%.**

- **Contextual Precision: 85.0%.**
- **Pass/Fail:** 12 Passed, 3 Failed.

Experiment 5: Hyperparameter Tuning ((K=3))

- **Action:** Tested reducing top-(K) from 5 to 3 to evaluate if removing lower ranked chunks improves precision.
- **Results:** Contextual Precision settled at **84.0%** due to minor sample variance across our 15 case golden dataset.

Final Operational Benchmark Summary

Experiment Run	Configuration Changes	Contextual Recall	Contextual Precision	Failed Cases
Run 1 (Baseline)	750/100, small embeddings, K = 5, No Re-ranker	80.0%	80.0%	5 / 15
Run 2	1000/150, small embeddings, K = 5, No Re-ranker	97.0%	83.0%	3 / 15
Run 3	1000/150, small embeddings, K = 5, Cross Encoder Re-ranker	92.0%	85.0%	2 / 15
Run 4	1000/150, large embeddings, K = 5, Re-ranker	99.0%	85.0%	3 / 15
Run 5	1000/150, large embeddings, K = 3, Re-ranker	99.0%	84.0%	3 / 15

Through systematic evaluation and component tuning, we raised our retrieval performance from an 80/80 baseline to achieving **95% Contextual Recall** and **~85% Contextual Precision**. We can now move forward with complete quantitative confidence in our retriever component to build and evaluate the Generator module.